Date: 16/07/2026 **Week No.: 2** **Program S. No. 4**

<table>
<tr><th>Title</th><th>Data Manipulation and Analysis using the Starwars Dataset in R</th></tr>
<tr><td>Problem Statement</td><td>A streaming platform maintains movie records containing:
• MovieID
• Rating
• Genre
Perform:
1. Create a dataset with missing values.
2. Fill missing Rating values using average rating.
3. Remove rows with missing MovieID or Genre.
4. Convert Rating into integer type.
5. Normalize rating values.</td></tr>
<tr><td>Objectives</td><td>• To understand movie data preprocessing using the Pandas library.
• To create a dataset containing missing values.
• To identify and handle missing values in a dataset.
• To fill missing Rating values using the average rating.
• To remove records with missing MovieID or Genre.
• To convert the Rating column into integer data type.
• To normalize movie rating values using Min-Max normalization.
• To identify top-rated movies based on ratings.</td></tr>
<tr><td>Dataset URL Link / File</td><td>
<table>
<tr><th>MovieID</th><th>MovieTitle</th><th>Rating</th><th>Genre</th><th>ViewCount</th><th>Likes</th></tr>
<tr><td>MV101</td><td>The Last Horizon</td><td>8.5</td><td>Science Fiction</td><td>125000</td><td>11200</td></tr>
<tr><td>MV102</td><td>City Lights</td><td></td><td>Comedy</td><td>98000</td><td>7500</td></tr>
<tr><td>MV103</td><td>Hidden Truth</td><td>7.8</td><td>Thriller</td><td>110000</td><td>8900</td></tr>
<tr><td>None</td><td>Lost Kingdom</td><td>9.1</td><td>Fantasy</td><td>150000</td><td>13500</td></tr>
<tr><td>MV105</td><td>Digital Dreams</td><td>8.9</td><td>Science Fiction</td><td>175000</td><td>16000</td></tr>
<tr><td>MV106</td><td>Silent River</td><td></td><td>Drama</td><td>87000</td><td>6100</td></tr>
<tr><td>MV107</td><td>Future Code</td><td>9.3</td><td>Science Fiction</td><td>210000</td><td>19800</td></tr>
<tr><td>MV108</td><td>Broken Compass</td><td>6.7</td><td></td><td>65000</td><td>4200</td></tr>
<tr><td>MV109</td><td>Midnight Journey</td><td>8.1</td><td>Adventure</td><td>132000</td><td>10400</td></tr>
<tr><td>MV110</td><td>Laugh Again</td><td>7.5</td><td>Comedy</td><td>105000</td><td>8100</td></tr>
<tr><td>MV111</td><td>Ocean Mystery</td><td>8.8</td><td>Mystery</td><td>143000</td><td>12100</td></tr>
<tr><td>MV112</td><td>Final Mission</td><td></td><td>Action</td><td>190000</td><td>17400</td></tr>
</table>
</td></tr>
</table>

MV113	Parallel Lives	8.4	Drama	118000	9600
MV114	Wild Trails	7.2	Adventure	99000	7200
MV115	The Great Escape	9.0	Action	205000	18800

Functions Details

Sl. No.	Function	Package	Description of function
1	pd.DataFrame()	Pandas	Creates a DataFrame from the movie dataset stored in dictionary format.
2	isnull()	Pandas	Identifies missing values present in the movie dataset.
3	mean()	Pandas	Fills missing Rating values using the calculated average rating.
4	dropna()	Pandas	Removes rows containing missing MovieID or Genre values.
5	pd.to_numeric()	Pandas	Converts Rating values into numeric format and handles invalid entries.
6	round()	Pandas	Rounds decimal Rating values before converting them into integers.
7	astype()	Pandas	Converts the Rating column into integer data type.
8	min()	Pandas	Finds the minimum Rating value required for normalization.
9	max()	Pandas	Finds the maximum Rating value required for normalization.
10	groupby()	Pandas	Groups movie records according to Genre for genre-wise analysis.
11	sort_values()	Pandas	Sorts movies based on rating, popularity score or number of views.
12	fillna()	Pandas	Fills missing Rating values using the calculated average rating.

Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model

Step 1: Import the Pandas library and create the movie dataset in dictionary format.

Step 2: Convert the dictionary into a Pandas DataFrame using pd.DataFrame().

Step 3: Display the original dataset containing missing values.

Step 4: Calculate the average value of the Rating column using the mean() function.

Step 5: Fill the missing Rating values using fillna() with the calculated average rating.

	Step 6: Remove rows containing missing MovieID or Genre using the dropna() function. Step 7: Convert the Rating column into numeric format using pd.to_numeric() and then convert it into integer type using round() and astype(int). Step 8: Normalize the Rating values using the Min-Max Normalization formula: Normalized Rating = (Rating − Minimum Rating) / (Maximum Rating − Minimum Rating) Step 9: Calculate the Engagement Rate and Popularity Score for each movie based on views, likes and ratings. Step 10: Sort the movies according to Popularity Score and Rating using sort_values() and assign popularity ranks. Step 11: Generate a genre-wise report using the groupby() function to compare movie performance across different genres. Step 12: Recommend the top movies based on selected genre and minimum rating, generate the automated analytics report, and save the processed results for future analysis.
Program	(code below)

```
import pandas as pd


# ----------------------------------------------------------
# 1. CREATE MOVIE DATASET IN DICTIONARY FORMAT
# ----------------------------------------------------------
movie_data = {
    "MovieID": [
        "MV101", "MV102", "MV103", None, "MV105",
        "MV106", "MV107", "MV108", "MV109", "MV110",
        "MV111", "MV112", "MV113", "MV114", "MV115"
    ],

    "MovieTitle": [
        "The Last Horizon",
        "City Lights",
        "Hidden Truth",
        "Lost Kingdom",
        "Digital Dreams",
        "Silent River",
        "Future Code",
        "Broken Compass",
        "Midnight Journey",
```

```
        "Laugh Again",
        "Ocean Mystery",
        "Final Mission",
        "Parallel Lives",
        "Wild Trails",
        "The Great Escape"
    ],

    "Rating": [
        8.5, None, 7.8, 9.1, 8.9,
        None, 9.3, 6.7, 8.1, 7.5,
        8.8, None, 8.4, 7.2, 9.0
    ],

    "Genre": [
        "Science Fiction",
        "Comedy",
        "Thriller",
        "Fantasy",
        "Science Fiction",
        "Drama",
        "Science Fiction",
        None,
        "Adventure",
        "Comedy",
        "Mystery",
        "Action",
        "Drama",
        "Adventure",
        "Action"
    ],

    "ViewCount": [
        125000, 98000, 110000, 150000, 175000,
        87000, 210000, 65000, 132000, 105000,
        143000, 190000, 118000, 99000, 205000
    ],

    "Likes": [
        11200, 7500, 8900, 13500, 16000,
        6100, 19800, 4200, 10400, 8100,
        12100, 17400, 9600, 7200, 18800
    ]
}


# ----------------------------------------------------------
# 2. CONVERT DICTIONARY INTO DATAFRAME
```

```
# ------------------------------------------------------------
movies = pd.DataFrame(movie_data)

print("\nORIGINAL MOVIE DATASET")
print("=" * 90)
print(movies.to_string(index=False))


# ------------------------------------------------------------
# 3. CALCULATE AVERAGE RATING
# ------------------------------------------------------------
average_rating = movies["Rating"].mean()

print("\nAVERAGE RATING")
print("=" * 90)
print("Average Rating:", round(average_rating, 2))


# ------------------------------------------------------------
# 4. FILL MISSING RATING VALUES
# ------------------------------------------------------------
movies["Rating"] = movies["Rating"].fillna(average_rating)

print("\nDATASET AFTER FILLING MISSING RATING VALUES")
print("=" * 90)
print(
    movies[
        ["MovieID", "MovieTitle", "Rating", "Genre"]
    ].to_string(index=False)
)


# ------------------------------------------------------------
# 5. REMOVE ROWS WITH MISSING MOVIEID OR GENRE
# ------------------------------------------------------------
movies = movies.dropna(subset=["MovieID", "Genre"])

print("\nDATASET AFTER REMOVING MISSING MOVIEID OR
GENRE")
print("=" * 90)
print(
    movies[
        ["MovieID", "MovieTitle", "Rating", "Genre"]
    ].to_string(index=False)
)


# ------------------------------------------------------------
```

```
# 6. CONVERT RATING INTO INTEGER TYPE
# ----------------------------------------------------------
movies["Rating"] = movies["Rating"].round().astype(int)

print("\nRATING CONVERTED INTO INTEGER TYPE")
print("=" * 90)
print(
    movies[
        ["MovieID", "MovieTitle", "Rating"]
    ].to_string(index=False)
)

print("\nRating Data Type:", movies["Rating"].dtype)


# ----------------------------------------------------------
# 7. NORMALIZE RATING VALUES
# Min-Max Normalization
# ----------------------------------------------------------
minimum_rating = movies["Rating"].min()
maximum_rating = movies["Rating"].max()

movies["NormalizedRating"] = (
    (movies["Rating"] - minimum_rating)
    / (maximum_rating - minimum_rating)
).round(3)

print("\nNORMALIZED RATING VALUES")
print("=" * 90)
print(
    movies[
        [
            "MovieID",
            "MovieTitle",
            "Rating",
            "NormalizedRating"
        ]
    ].to_string(index=False)
)


# ----------------------------------------------------------
# 8. CALCULATE ENGAGEMENT RATE
# ----------------------------------------------------------
movies["EngagementRate"] = (
    movies["Likes"] / movies["ViewCount"] * 100
).round(2)
```

```
print("\nVIEWER ENGAGEMENT")
print("=" * 90)
print(
    movies[
        [
            "MovieTitle",
            "ViewCount",
            "Likes",
            "EngagementRate"
        ]
    ].to_string(index=False)
)


# ------------------------------------------------------------
# 9. CALCULATE POPULARITY SCORE
# ------------------------------------------------------------
maximum_engagement = movies["EngagementRate"].max()

movies["NormalizedEngagement"] = (
    movies["EngagementRate"] / maximum_engagement
).round(3)

movies["PopularityScore"] = (
    movies["NormalizedRating"] * 0.60
    + movies["NormalizedEngagement"] * 0.40
).round(3)


# ------------------------------------------------------------
# 10. SORT AND RANK MOVIES
# ------------------------------------------------------------
movies = movies.sort_values(
    by=["PopularityScore", "Rating"],
    ascending=[False, False]
).reset_index(drop=True)

movies["PopularityRank"] = range(1, len(movies) + 1)

print("\nTOP-RATED AND POPULAR MOVIES")
print("=" * 90)
print(
    movies[
        [
            "PopularityRank",
            "MovieID",
            "MovieTitle",
            "Genre",
```

```
            "Rating",
            "ViewCount",
            "PopularityScore"
        ]
    ].to_string(index=False)
)


# ------------------------------------------------------------
# 11. GENERATE GENRE-WISE REPORT
# ------------------------------------------------------------
genre_report = movies.groupby(
    "Genre",
    as_index=False
).agg(
    MovieCount=("MovieID", "count"),
    AverageRating=("Rating", "mean"),
    HighestRating=("Rating", "max"),
    TotalViews=("ViewCount", "sum"),
    TotalLikes=("Likes", "sum")
)

genre_report["AverageRating"] = (
    genre_report["AverageRating"].round(2)
)

genre_report = genre_report.sort_values(
    by="AverageRating",
    ascending=False
)

print("\nGENRE-WISE MOVIE REPORT")
print("=" * 90)
print(genre_report.to_string(index=False))


# ------------------------------------------------------------
# 12. SIMPLE MOVIE RECOMMENDATION
# ------------------------------------------------------------
preferred_genre = "Science Fiction"
minimum_required_rating = 7

recommended_movies = movies[
    (movies["Genre"] == preferred_genre)
    & (movies["Rating"] >= minimum_required_rating)
].sort_values(
    by="PopularityScore",
    ascending=False
```

```
)

print("\nMOVIE RECOMMENDATIONS")
print("=" * 90)
print("Preferred Genre:", preferred_genre)
print("Minimum Rating:", minimum_required_rating)

print(
    recommended_movies[
        [
            "MovieID",
            "MovieTitle",
            "Genre",
            "Rating",
            "PopularityScore"
        ]
    ].to_string(index=False)
)


# ------------------------------------------------------------
# 13. AUTOMATED ANALYTICS REPORT
# ------------------------------------------------------------
top_movie = movies.iloc[0]

most_viewed_movie = movies.loc[
    movies["ViewCount"].idxmax()
]

best_genre = genre_report.iloc[0]

print("\nAUTOMATED MOVIE ANALYTICS REPORT")
print("=" * 90)

print("Total Movies Analysed:", len(movies))
print("Average Movie Rating:", round(movies["Rating"].mean(), 2))
print("Total Genres:", movies["Genre"].nunique())

print("\nTop-Rated and Most Popular Movie")
print("Movie Title:", top_movie["MovieTitle"])
print("Genre:", top_movie["Genre"])
print("Rating:", top_movie["Rating"])
print("Popularity Score:", top_movie["PopularityScore"])

print("\nMost Viewed Movie")
print("Movie Title:", most_viewed_movie["MovieTitle"])
print("Views:", most_viewed_movie["ViewCount"])
```

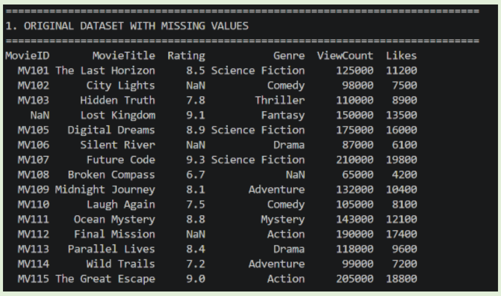
```
print("\nBest Performing Genre")
print("Genre:", best_genre["Genre"])
print("Average Rating:", best_genre["AverageRating"])


# -----------------------------------------------------------
# 14. SAVE RESULTS
# -----------------------------------------------------------
movies.to_csv(
    "processed_movie_data.csv",
    index=False
)

genre_report.to_csv(
    "genre_wise_report.csv",
    index=False
)

print("\nFILES CREATED")
print("=" * 90)
print("processed_movie_data.csv")
print("genre_wise_report.csv")
```

Output Snapshots With Explanation/ Observations

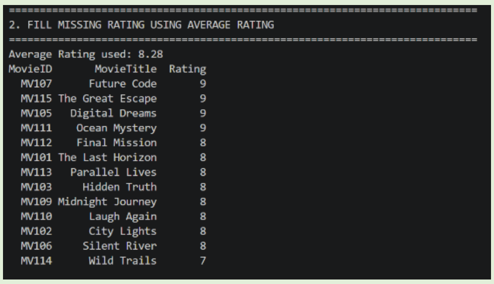


```
==========================================================================
1. ORIGINAL DATASET WITH MISSING VALUES
==========================================================================
MovieID       MovieTitle  Rating           Genre  ViewCount  Likes
  MV101 The Last Horizon     8.5 Science Fiction     125000  11200
  MV102      City Lights     NaN          Comedy      98000   7500
  MV103     Hidden Truth     7.8        Thriller     110000   8900
    NaN     Lost Kingdom     9.1         Fantasy     150000  13500
  MV105   Digital Dreams     8.9 Science Fiction     175000  16000
  MV106     Silent River     NaN           Drama      87000   6100
  MV107      Future Code     9.3 Science Fiction     210000  19800
  MV108   Broken Compass     6.7             NaN      65000   4200
  MV109 Midnight Journey     8.1       Adventure     132000  10400
  MV110      Laugh Again     7.5          Comedy     105000   8100
  MV111    Ocean Mystery     8.8         Mystery     143000  12100
  MV112    Final Mission     NaN          Action     190000  17400
  MV113   Parallel Lives     8.4           Drama     118000   9600
  MV114      Wild Trails     7.2       Adventure      99000   7200
  MV115 The Great Escape     9.0          Action     205000  18800
```

Figure 1.1 Original dataset printed after creating the DataFrame (containing missing values).

The movie dataset was successfully created in DataFrame format with missing values in the **MovieID**, **Rating**, and **Genre** columns.

The average rating was calculated successfully and was used to replace all missing Rating values.



```
==========================================================================
2. FILL MISSING RATING USING AVERAGE RATING
==========================================================================
Average Rating used: 8.28
MovieID       MovieTitle  Rating
  MV107      Future Code       9
  MV115 The Great Escape       9
  MV105   Digital Dreams       9
  MV111    Ocean Mystery       9
  MV112    Final Mission       8
  MV101 The Last Horizon       8
  MV113   Parallel Lives       8
  MV103     Hidden Truth       8
  MV109 Midnight Journey       8
  MV110      Laugh Again       8
  MV102      City Lights       8
  MV106     Silent River       8
  MV114      Wild Trails       7
```

Figure 1.2 Output showing the calculated average rating.

```
================================================================================
3. REMOVE ROWS WITH MISSING MOVIEID OR GENRE
================================================================================
MovieID       MovieTitle           Genre  Rating
  MV107      Future Code Science Fiction       9
  MV115 The Great Escape          Action       9
  MV105   Digital Dreams Science Fiction       9
  MV111     Ocean Mystery        Mystery       9
  MV112     Final Mission         Action       8
  MV101 The Last Horizon Science Fiction       8
  MV113    Parallel Lives          Drama       8
  MV103      Hidden Truth       Thriller       8
  MV109 Midnight Journey       Adventure       8
  MV110       Laugh Again         Comedy       8
  MV102       City Lights         Comedy       8
  MV106      Silent River          Drama       8
  MV114       Wild Trails      Adventure       7
```

Figure 1.3 Dataset after fillna() operation.

Missing Rating values were replaced with the calculated average rating while preserving all other records.

Rating values were converted into integer type and normalized using the Min-Max Normalization technique.

```
================================================================================
5. NORMALIZE RATING VALUES
================================================================================
MovieID       MovieTitle  Rating  NormalizedRating
  MV107      Future Code       9               1.0
  MV115 The Great Escape       9               1.0
  MV105   Digital Dreams       9               1.0
  MV111    Ocean Mystery       9               1.0
  MV112    Final Mission       8               0.5
  MV101 The Last Horizon       8               0.5
  MV113   Parallel Lives       8               0.5
  MV103     Hidden Truth       8               0.5
  MV109 Midnight Journey       8               0.5
  MV110      Laugh Again       8               0.5
  MV102      City Lights       8               0.5
  MV106     Silent River       8               0.5
  MV114      Wild Trails       7               0.0
```

*Figure 1.4 Integer Rating output along with the **NormalizedRating** column.*

```
================================================================================
TOP-RATED AND POPULAR MOVIES
================================================================================
 PopularityRank MovieID       MovieTitle           Genre  Rating  ViewCount  PopularityScore
              1   MV107      Future Code Science Fiction       9     210000            1.000
              2   MV115 The Great Escape          Action       9     205000            0.989
              3   MV105   Digital Dreams Science Fiction       9     175000            0.988
              4   MV111    Ocean Mystery         Mystery       9     143000            0.959
              5   MV112    Final Mission          Action       8     190000            0.689
              6   MV101 The Last Horizon Science Fiction       8     125000            0.680
              7   MV113   Parallel Lives           Drama       8     118000            0.645
              8   MV103     Hidden Truth        Thriller       8     110000            0.643
              9   MV109 Midnight Journey       Adventure       8     132000            0.634
             10   MV110      Laugh Again          Comedy       8     105000            0.627
             11   MV102      City Lights          Comedy       8      98000            0.624
             12   MV106     Silent River           Drama       8      87000            0.597
             13   MV114      Wild Trails       Adventure       7      99000            0.308
```

*Figure 1.5 Sorted movie list showing **Popularity Rank**.*

Movies were ranked successfully based on their popularity score, allowing easy identification of top-performing movies.

Conclusions

The Movie Rating Data Preprocessing project was successfully implemented using the Pandas library. The dataset was created with missing values, and the missing Rating values were replaced using the average rating. Records with missing MovieID or Genre were removed to improve data quality. The Rating column was converted into integer type, and the values were normalized using the Min-Max normalization technique. The project also identified top-rated movies, generated genre-wise reports, analysed viewer preferences, recommended

	movies based on genre and rating, and visualized movie popularity trends. This exercise provided practical knowledge of data preprocessing, data analysis, and dashboard development using Python, Pandas, and Streamlit.
References	1. Michael Freeman and Joel Ross, Programming Skills for Data Science: Start Writing Code to Wrangle, Analyze, and Visualize Data with R, Addison-Wesley, 2018 2. Python Program (https://github.com/hasnainmakada-99/Data-Science-Codes/blob/main/WEEK%202/EXERCISE_PROGRAM.py) 3. Class lecture link